ARTICLE | Article in Press

# A robust and reliable method for malware classification using hybrid features and gradient boosting

**Asia Maqsood · Hamid Turab Mirza · Aliza Muskan · Arslan Akram · Javed Rashid · Raed Alharthi · Nadeem Sarwar**

Received: 19 November 2025 / Accepted: 5 October 2026
© The Author(s) 2026

**Abstract**

The most significant problem facing software security firms in the current cybersecurity system is the fact that new strains of malware keep on appearing. The achievement of successful detection of malicious software based on both static and dynamic data remains unmet despite the further realization of different approaches to deep learning and machine learning. Defects in encryption, packaging, and distribution, the discrepancies between the distribution of malware samples in families, make this issue even worse. To do actual malware research that utilizes deep learning methods, you will require data scientists of appropriate competence, large amounts of computer power (particularly GPUs) and time. In order to overcome such issues, this approach involves a hybrid approach to feature extraction that integrates wavelet and other statistical features based on benchmarked malware image sets. A classification model based on gradient-boosting can be used reliably to classify different types of malwares. Further, this paper examined the capability of the machine learning system to Classify in-distribution test samples well. This method is properly tested on the two benchmarked datasets MelVis and MALIMG to ensure its effectiveness. By joining these two sets of data it is possible to test the suggested technique using varied array of samples. The proposed approach outperforms the state-of-the-art techniques with an incredible accuracy rate of 100% using MALIMG and 98.10 and 98.40% using combined datasets. Its value extends far beyond cybersecurity in such spheres as sustaining IoT ecosystems to assist in improving networked devices and systems.

**Keywords** Multiclass malware classification · GPU-free method · Gradient boosting · XGBOOST · Perturbation · Machine learning

## Introduction

Bad software may harm computer systems and structures. Malware attacks have become a significant menace to the security of computing due to the rapid proliferation. Malware has not only increased, but also subsequent discoveries have hinted out that various forms of malware have become more advanced[1]. Identifying the best approach to be applied in detecting malware is one of the most crucial activities in cybersecurity. A significant

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.



https://doi.org/10.1038/s41598-026-75059-1

Published online: 17 October 2026

scientific reports

number of antivirus engines have been found to utilize signature-based discovery in as far as malware detection strategies are concerned[2]. Space experts normally describe the marks, that consist of the design of the byte code and content strings, in their study of malware cases. After deciding on markers for malware detection, the following step is to harmonize their designs. Though signature matching is a powerful tool for detecting malware that changes over time, signature-based techniques can't keep up with the dynamic nature of those malware variants.

To address these problems, several machine learning methods have been suggested in the literature[3,4]. Unlike signature-based methods, machine learning endeavors to construct a data-driven method for malware classification based on described highlights. Two primary extraction strategies are used in malware analysis: energetic and inactive[5]. The inactive analysis happens in a non-runtime environment, and bypass construction enables the extraction of comparison highlights (such as byte-sequence and string highlights) from executables. To carry out the energetic evaluation, it is required to run the virus in a different environment so that its behavior can be monitored, and any malicious attacks may be documented. Space professionals may organize executable properties to detect malware based on active and passive study. A plethora of methods for malware detection have been suggested, some of which include extraction[6]. But getting these manual features out could be a pain and take a long time.

Many publications also used visual cues in various viral detection approaches in addition to the features stated by space scientists[7–9]. Consequently, Authors were prepared to communicate with each executable using visual cues like SURF, Essence[10], and Filter[11]. After extracting the visual highlights, many malicious executables can be categorized using conventional approaches, including support vector machines (SVM). On the flip side, convolutional neural networks (CNNs) have shown excellent computer vision tasks like protest detection and image classification. Convolutional neural networks (CNNs) are like computer vision applications in that they can learn channels and automatically extract informative highlights. Many malware identification approaches have also used CNNs as an end-to-end finder[9,12]. These studies aim to categorize malware executables created by parallel execution into families based on similarities between them[10–16].

One of the biggest problems in cybersecurity is finding malware and identifying it via image analysis. Traditional methods of malware detection have grown less effective due to the increasing sophistication of malware. Many issues have been found with the most current approaches. To start, the current methods could be better at extracting the most useful visual elements from binary malware images. Second, a large portion of the research community was obsessed with finding the bare minimum of malware families to categorize. Finally, there needs to be more diversity in the databases used to classify malware images. To address this issue, we need to create and deploy a Stable across folds machine learning method that can correctly classify malware families based on visual representations. Improving cybersecurity measures in response to ever-changing threats can be achieved, in part, by extracting valuable information from images of malware samples to enable scalable and Stable across folds categorization. Examining hostile attacks on the machine learning process is important for ensuring the malware classification method is resilient.

The novelty of this work lies in four specific contributions that distinguish it from prior malware image classification approaches. First, we introduce a weighted hybrid feature fusion strategy that combines GLCM textural features and Wavelet multi-resolution features using variance-based normalization, preventing high-dimensional GLCM features from dominating the lower-dimensional Wavelet features during XGBoost training. Second, we provide a systematic comparison of independent and combined dataset evaluation protocols with explicit family-wise overlap control, ensuring that no malware family appears in both training and testing sets across datasets. Third, we present a GPU-free, CPU-optimized pipeline with explicit efficiency-accuracy trade-off analysis, reporting training time, inference time, and memory footprint alongside classification accuracy. Fourth, we conduct comprehensive robustness evaluation via 10-fold cross-validation with per-class performance analysis, confusion matrices, and ROC-AUC curves, revealing class-specific detection patterns that are obscured in aggregate accuracy reporting. These contributions collectively advance the state of the art in resource-efficient, interpretable, and rigorously evaluated malware image classification.

1. A hybrid feature extraction approach that enhances machine learning algorithm resilience for multiclass classification by extracting the most relevant characteristics from malware images.
2. To address the problem of bias in the classification model's performance, the Synthetic Minority Over-sampling Technique has been employed to balance the dataset's samples.
3. Two datasets, MALIMG and Melvis, were utilized to test the multiclass classification performance of the Fine-Tuned Gradient Boosting technique.
4. An analysis of the proposed framework's performance in comparison to current best practices for malware image classification.

The parts in this article have an order of sequence. Part 2 is going to discuss the literature on malware detection methods in detail. In "Material and methods", we review the experimental databases and the method suggested. This methodology includes classification, feature extraction and preprocessing. "Results and analysis" outlines the experimental design and gives the results on the proposed architecture, supported by figures and tables. "Conclusion" presents the future research directions that can be projected on the findings of the study and brings the study to an end.

## Literature review

The literature review summarizes approaches and models of classification of malware in different datasets. Using a wide variety of classes (Adrd, BaseBridge, DroidDream, and others)[17] has achieved 97% accuracy in identifying trusted classes and malware with the help of machine learning. Likewise[18] employed self-gathered data with the help of k-nearest Neighbors with GIST descriptors and the random Forest classifiers to achieve 97% precision. A convolutional neural network (CNN) classifier for differentiating between the dangerous and non-dangerous classes has a success rate of 92.8 percent when it is trained using the Malimg dataset. Static, dynamic and vision-based analysis was also investigated using datasets of Malimg, BIG 2015, and MalVis[17], the corresponding accuracies were 98.65, 97.2 and 97.43. In Ref.[18], the authors applied Random Forest, Naive Bayes and SVM to VXheaven, Nothing, and VirusShare data using a ten-fold cross-validation. Three different levels of precision were reported (99.11, 97.68 and 96.8). Obtained 97 percent accuracy on Malimg dataset with VGG19 CNN with spatial attention and convolutional neural network (SACNN)[19]. In addition, Ref.[20] also used deep-learning CNN classifiers with which VirusShare datasets were classified as harmless or harmful with a success rate of 93.01%.

On their own data, the authors of[21] achieved accuracies of 80%, 89 and 95 percent using the Haralick image texture features with K-nearest neighbor (KNN), naive Bayes (NB) and Random Forest (RF). Moreover, Ref.[22] achieved a 91.96 percent accuracy rate on the Drebin and Malgenom databases using multiple classifiers and features, such as SIFT, SURF, ORB, and KAZE[23] also used CNN, SVM, Decision Tree and Bagging algorithms on Drebin dataset and attained a very impressive accuracy rate of 96.8%. In conclusion, machine-learning algorithm applied by[24] to Derbin data consisting of CNN, k-Nearest Neighbor (KNN), Support Vector Machine (SVM), and Random Forest (RF). A 92.59 percent classification accuracy was obtained. In a study, research is presented that has enhanced accuracy of classifying malware in a variety of datasets by applying various methodologies and models. SVM, Random Forest (RF), and K-Nearest Neighbor (KNN) were also used in the Durbin dataset as a type of CNN-based machine learning algorithm. In terms of the proper differentiation of safe and dangerous classes, their strategy did it at the rate of 92.59.

In comparative study on Malimg dataset, it has been reported that ViT has an accuracy of 97.31% with CNN with accuracy 95.32% and CNN-SVM at 95.37% in 2025[25]. The same work demonstrates that attention-augmented CNN variants (SE-ResNet: 98.97%, CBAM-ResNet: 98.60%) as well as the proposed GCSA-ResNet (99.08%) set a new upper bound of performance that is not approached by the methods cited by the original review paper (e.g., 97% for SACNN). It's not just a step change in architecture. The self-attention mechanism is an innovative approach that overcomes the inability of CNNs to capture long-range dependencies between code

fragments spatially far from each other, a fundamental issue in malware visualization. In a study on ViT-based malware detection from 2025, researchers found that CNNs are unable to capture global context, making them less Stable across folds to traffic mimicking malware, while the Transformer backbones learn global dependencies and high level-of-feature representations from patch embeddings c. The "Through the Static" study (2025) shows that explainability insights from CNN decisions can be transferred to improve the performance of Visual Transformers, achieving F1 score improvements between 2–8% on Malimg, BIG2015 and VX-Zoo. This is a methodologically innovative approach, one that the literature cited by the original paper doesn't foresee the possibility of interpretation as a post hoc diagnostic, but as a training signal for architectural refinement[26].

While the authors of AMALGAN (2025) only use GANs for data augmentation, they use the GAN discriminator as an auxiliary classifier. The framework outperforms with 95.24% accuracy, 94.31% precision, 96.12% recall and 94.11% F1-score for the image based android malware classification. The paradigm (generative components as detectors instead of training tools) is also conceptually different from the classification of approaches in the review[27]. In an independent 2025 study, the authors use a modified DCGAN and ResNet to generate augmentation on Malimg which, together with a classifier of MobileNetV3, reaches 96% accuracy, 98% precision, and 98% F1-score. While the review did not comment on generative methods, there is a significant omission there[28]. The state-of-the-art methods are comparatively analyzed in Table 1.

Lastly, this is evidenced by extensive literature that indicates the multi-methodology that is used in malware classification. This has been made possible by use of sophisticated feature extraction systems such as GIST descriptors and HARALICK image texture features which have enhanced the accuracy in identification of hostile entities. It has also been contributed by machine learning algorithms such as support vector machines and the k-nearest neighbors. The effectiveness of these procedures can be considered greater than 90 percent. These

**Table 1** Comparison of state-of-the-art methods.

| Ref, year | Methodology | Dataset name | Classes | Results |
|---|---|---|---|---|
| [17], 2022 | Machine learning | Multiple Datasets | Binary | 97 |
| [18], 2020 | GIST descriptors + Different Machine learning algorithms | Self-Collected | Binary | 97 |
| [19], 2020 | CNN | Malimg MSIC | Binary | 92.8 |
| [20], 2020 | static analysis, dynamic analysis, vision-based analysis | Malimg, BIG 2015, MalVis | Binary | 98.65, 97.2, 97.43 |
| [21], 2021 | Naïve byes, SVM, and Random Forest | VXheaven, Nothing, VirusShare | Binary | 96.8, 97.68, 99.11 |
| [22], 2021 | Spatial attention and convolutional neural network (SACNN) | Malimg dataset | Binary | 97 |
| [23], 2020 | CNN | VirusShare | Binary | 93.01 |
| [24], 2020 | Haralick image texture features, K-nearest neighbor (KNN), naïve Bayes (NB), Random forest (RF) | Self-collected | Binary | 80, 89, 95 |
| [25], 2025 | CNN-SVM | Malimg | Multiclass | 95.37 |
| [29], 2026 | HERL-ViT | Malware Data | Multiclass | 99.2 |
| [26], 2025 | Explainability-enhanced Visual Transformer | Malimg | Multiclass | 95.47 |
| [27], 2025 | AMALGAN | Android Malware Dataset | Binary | 95.24 |
| [28], 2025 | Modified DCGAN + ResNet + MobileNetV3 | Malimg | Multiclass | 96 |
| [30], 2021 | SIFT, SURF, ORB, KAZE and RF, K-NN, DT, Bagging AdaBoost, Gboost Voting | Drebin Malgenom | Binary | 91.96 |
| [31], 2023 | CNN, SVM, Decision Tree, Bagging | Drebin | Binary | 96.8 |
| [32], 2020 | CNN Features with K-Nearest Neighbor (KNN), Support Vector Machine (SVM), Random Forest (RF) | Derbin | Binary | 92.59 |

combined efforts underscore the necessity of strong approaches to strengthen cybersecurity controls in view of the dynamism of malware attacks.

## Material and methods

The proposed malware classification framework is evaluated on two distinct datasets, henceforth referred to as Dataset A and Dataset B. Both datasets undergo an identical preprocessing pipeline in which binary images are converted to grayscale representations. However, this shared preprocessing step does not imply dataset merging. Each dataset is processed, trained, and evaluated independently using the same XGBoost classifier and hybrid feature extraction approach (GLCM + Wavelet transformation). This independent evaluation protocol ensures that performance metrics reflect the model's generalizability within each dataset rather than across a combined corpus. Figure 1 illustrates the unified preprocessing pipeline and the subsequent independent classification workflow for each dataset.

### Datasets

Malware binaries are converted into grayscale images. In this study, images that have already been transformed are used. Datasets that are being used in this study are images that have already been converted from binaries of malware into grayscale images. The developers of datasets create a two-dimensional grayscale image of the malware file's bytes sequence. We can employ pattern recognition and image analysis for malware identification and classification by feeding these images into algorithms that process images and apply machine learning. Malware classification systems need this method to graphically represent malware family patterns and structures, which binary analysis cannot do. Classifiers can employ structural and textural qualities to differentiate safe and harmful files and even group malware into families by converting infected binaries into images. This conversion approach enhances malware detection by combining signature-based and heuristic analysis with image-based classification algorithms.

As mentioned earlier, this study provides a comprehensive investigation of malware classification using images. This is why we've used two separate datasets to measure how well the plan worked. In this study, samples were allocated to their respective families, and the system was evaluated for malware family classification. The two datasets have various kinds of malware samples, and this makes it better to categorize. In succeeding sections, we shall discuss the datasets in detail. The list of all the datasets and the categories on which they can be classified that are used in classification purposes in the current study are given in Table 2.

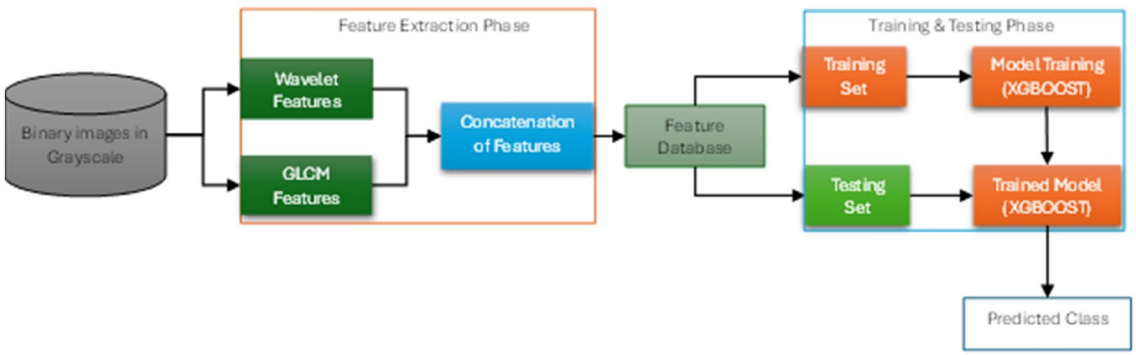



**Fig. 1** Workflow for proposed malware classification

**Table 2** Details of datasets used for experiments.

| Dataset name | Malware family | Malware name | No. of samples |
|---|---|---|---|
| MALIMG | Backdoor | AGENT.FYI and RBOT! GEN | 274 |
| | Others | SWIZZOR.GEN! I, LOLYDA.AA1, FAKEREAN, LOLYDA.AA3, DIALPLATFORM.B, LOLYDA.AA2, SWIZZOR.GEN! E, DONTOVO.A, WINTRIM.BX, ADIALER.C, RBOT! GEN, and ISTANTACCESS | 2451 |
| | Trojan | ALUERON.GEN! J, C2LOP.GEN! G, MALEX.GEN! J, SKINTRIM.N, and C2LOP.P | 760 |
| | Worm | ALLLAPLE.A, AUTORUN.K, VVB.AT, YUNER.A, and ALLLAPLE.L | 5854 |
| Total | | | 9339 |
| MalVis | Backdoor | ANDROM | 500 |
| | Others | ADPOSHEL, BROWSEFOX, INSTALLCORE, NEOREKLAMI, EXPIRO, NESHTA, SALITY, HACKKMS, VBA, AMONETIZE, and MULTIPLUG | 7311 |
| | Trojan | AGENT, ELAX, FASONG, SNARASITE, STANTINKO, VILSEL, REGRUN, DINWOD, INJECTOR, and VBKRYPT | 4941 |
| | Worm | ALLAPLE, AUTORUN, and HLUX | 1474 |
| Total | | | 14,226 |

### MALIMG

Researchers in the machine learning field and cybersecurity fields rely on the huge and critical Malimg dataset[33]. Its large collection of grayscale images of the various malware species makes it a good tool to train and test malware-identifying and malware-classifying machine learning models. The grayscale images included in this dataset are of 25 types of malwares. The extensive and elaborate Malimg dataset has become an invaluable resource in malware detection systems development and testing in the struggle against improved cybersecurity. The data set classification details have been given in Fig. 2.

### MelVis

The Malware Evaluation with Vision (MalwareVis) dataset[34] was utilized to assess the recommendation's effectiveness. According to Table 2, the MalVis dataset has 14,226 RGB byte images in total. There is a total of 26 distinct malware classes represented in these photographs; 25 of these images include malware, and 1 is cleanware. There were 5126 test samples and 9100 training samples among the 14,226 RGB byte images. An equal number of images from each class makes 350 in total. Figure 3 displays the various malware types found.

## Feature extraction

### Wavelet features

Wavelet transform is a mathematical method which has many applications in signal processing, data compression and image analysis. It begins with a wave and applies the situation to define a group of wavelets, the wavelets being smaller, transform versions of the first. The wavelet algorithm can be used to transform any signal, in either the continuous or discrete time. Discrete wavelet transformations (DWT) are often used in image processing in order to extract and compress features. The DWT is based on the coefficients that represent varying degrees of information and approximation to break down an image. These coefficients are obtained out of the image through a process known as convolving the image with a series of filters known as wavelet filters. The mathematical expression of the DWT is given as:

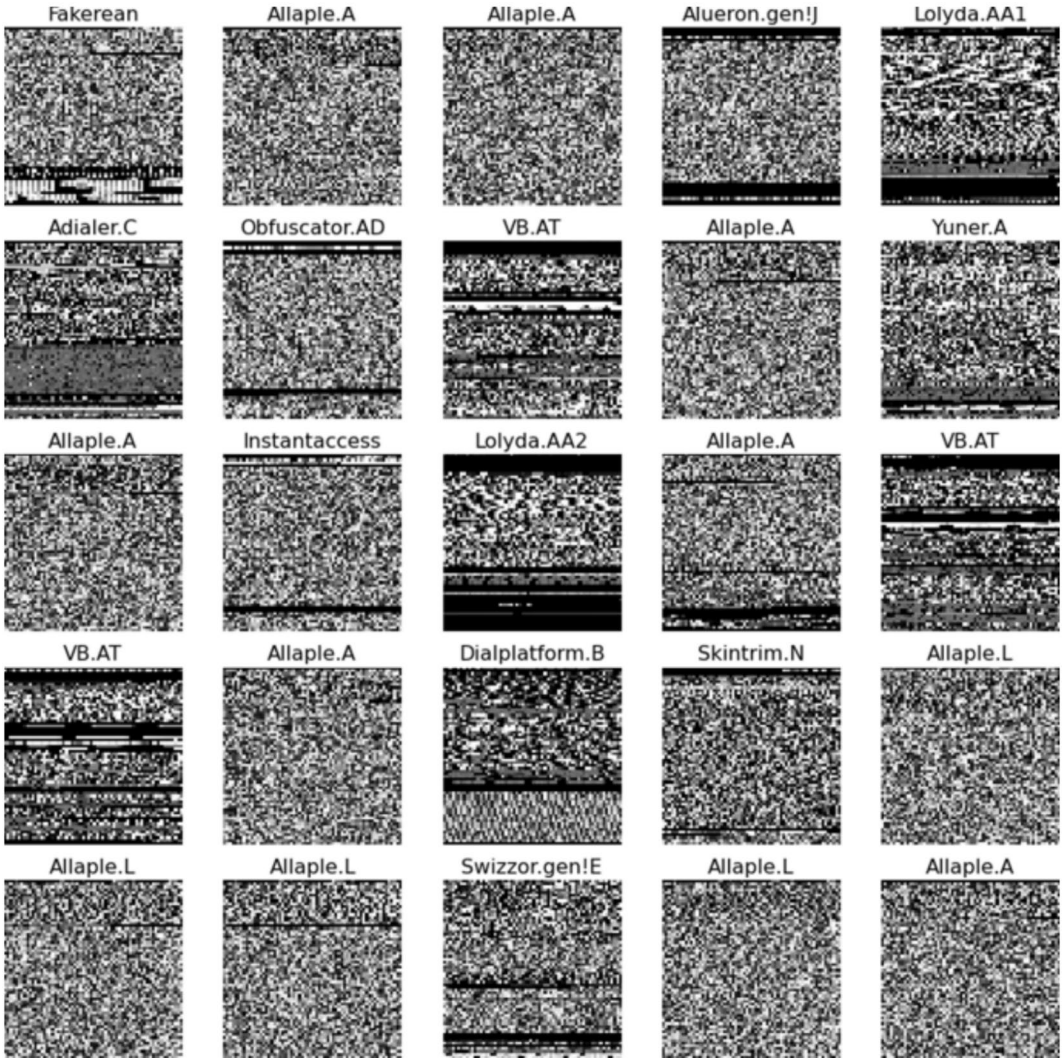



**Fig. 2** Sample images from MALIMG dataset.

$$W_{j,k} = \sum_{n=0}^{n-1} x_n \varphi_{j,k,n} \tag{1}$$

$$V_{j+1,k} = W_{j,k} = \sum_{n=0}^{n-1} x_n \emptyset_{j,k,n} \tag{2}$$

where $\varphi_{j,k,n}$ and $\emptyset_{j,k,n}$ are wavelet and scaling functions, and $x_n$ is the prime signal. At stage j and index number k, the wavelet and scaling constants are expressed by $W_{j,k}$ and $V_{j+1,k}$.

### Statistical features

One of the critical methods of recognizing patterns and image processing is feature extraction through GLCM (Gray-Level Co-occurrence Matrix). GLCM can be used to measure pixel intensities and thus capture spatial interdependence and textual information in images. Visual patterns are revealed subtly though significantly through the use of spatial linkages and intensity values to calculate the frequency of the pixel pairs. GLCM can use texture properties such as homogeneity, correlation, entropy, and contrast to extract the fine information

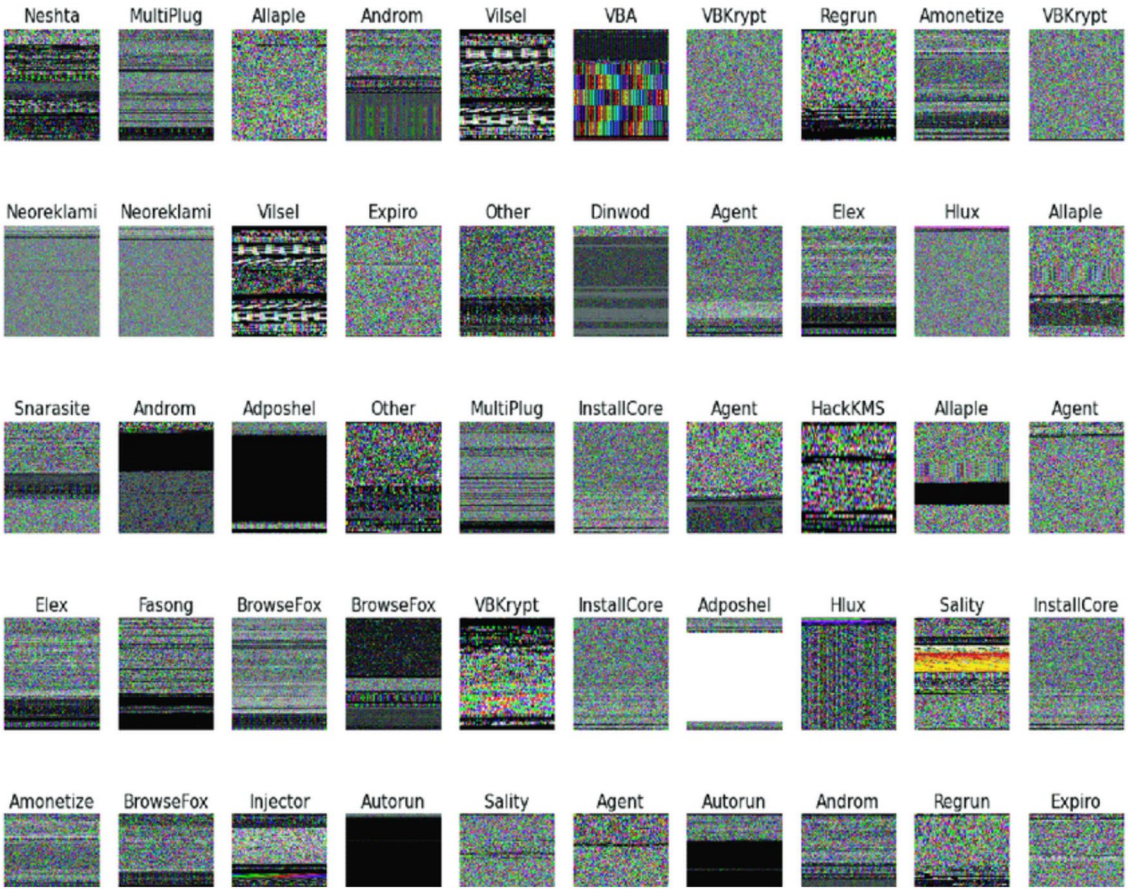



**Fig. 3** Sample images from MalVis dataset.

necessary to distinguish between textures and structures in images. To comprehend complex visual properties, extracting rich and discriminative features is necessary. These characteristics have applications in cybersecurity (namely, in analyzing and categorizing malware images), remote sensing, and medical imaging[35]. Like the other essential aspects, the useful statistical line formulas are as follows:

By accumulating the characteristics of all the pixel intensities, the intensity of each pixel is separated by the cumulative variety of image pixels. The average of an image is calculated as:

$$M = \left(\frac{1}{m} * n\right) \sum_{x=0}^{m_1} \sum_{y=0}^{n_1} f(x.y) \tag{3}$$

Second, and most importantly, consider the standard deviation, representing the observed population's probability distribution. On top of that, it can be used to gauge how dispersed the population is under observation. An image's intensity level and the promotion of the high end of a border of an image are indicated by a larger standard deviation value[36]. To calculate it, use the formula given below:

$$S.D\,(\sigma) = \sqrt{\left(\frac{1}{m * n}\right) \sum_{x=0}^{m_1} \sum_{y=0}^{n_1} (f(x.y) - M)^2} \tag{4}$$

Entropy must be included in a measure of image disorder of composition or unpredictability. Learning the distribution of pixel intensities of a image is a critical concept in information theory, especially in estimations of its uncertainty. As mentioned in reference[37], the entropy is determined by the complexity of the data visualization. This metric exposes visual complexity, and it measures the uncertainty of pixel values. An image with a smaller

entropy value has a more uniform or predictable distribution of pixel intensities, and an image with a larger value doesn't have much chaos or diversity. This type of entropies classification simplifies the process of locating structural complexity and non-linear patterns in images to a great extent. It allows simplified image processing, pattern recognition and data extraction, and further analysis.

$$\varepsilon = \sum_{x=0}^{m_1} \sum_{y=0}^{n_1} f(x.y) \log 2 f(x.y) \tag{5}$$

The skewness has been used as a measurement to understand whether there is a little absence of symmetry or an absolute absence of symmetry[38]. With the X probability distribution, the skewness is represented by the symbol Sk(x), which can be described in the following way:

$$Sk(x) = \left\{ \frac{1}{m} * n \right\} \frac{\sum \{F(x,y) - m\}\, 3}{S.D^3} \tag{6}$$

A Kurtosis is a parameter which is used to explain the form of the probability distributions of a random variable[39]. Along with its representation as the following, there is the kurtosis of a random variable which is also known as the Kurt of variable x.

$$Kurt(x) = \left\{ \frac{.1}{m} * n \right\} \frac{\sum (F(x,y) - m)4|}{S.D^4} \tag{7}$$

The variable known as energy is responsible for describing the quantifiable value that pertains to the scope of iterations of the image of pixels for the image. A parameter known as energy has been utilized to study the similarity between two images. Energy is defined by the Horalicks' interpretation of the G.L.C.M.'s features[40]. It is also known as the second angular moment, and as a result, it can be identified in the following manner:

$$En = \sqrt{\sum_{x=0}^{m_1} \sum_{y=0}^{n_1} F2(x.y)} \tag{8}$$

Using pixel-neighborhood relationships, one may quantify image structural nuances and textural variations. One of the most important ways to quantify these distinctions is through contrast. It measures the intensity difference involving a pixel and its neighboring pixels. In a small image area, contrast measures how varied the lights are[41]. The square of the difference of the intensity of the hub pixel and its neighbors. This statistic is an emphasis on image patterns, textures, sharp edges, and transitioning. Image processing, computer vision and the challenging task of detecting small differences in malware image analysis on the cybersecurity application of feature mining and image analysis uses contrast measurement. It allows the separation of small details and differences.

$$Con = \sum_{x=0}^{m_1} \sum_{y=0}^{n_1} (x_{y)^2 f(x.y)}) \tag{9}$$

The localized assimilation of an image is, once this is achieved, strong-minded by the opposite moments of differential, popularly referred to as I.D.M. To control whether the entire image has been textured lusterlessly, the I.D.M. scheme may stream a convinced number, or a collection of values, to control the scheme[42].

$$I.D.M = \sum_{x=0}^{m_1} \sum_{y=0}^{n_1} \frac{1}{1 + (x_y)\, 2} f\{x.y\} \tag{10}$$

Directional moments are a crucial statistic when it comes to evaluating the alignment of a image with the angle[43]. Some of the examples define and communicate some visual qualities, which involve nuances of color and the abundance of texture. This directional moment calculation is used to quantify the orientation of the image, and structural patterns and subtle changes at different angles can be captured. The technique ensures that there are alignment and distribution of the image features with regard to angles. Measuring these directional properties, it is easy to find out underlying structural layout of the image, and it can be said that it is easy to extract texture-based

features that can be used in object recognition, texture synthesis and, most importantly, cybersecurity of malware image patterns and classification.

$$D.M = \sum_{x_i=0}^{m-1} \sum_{y_i=0}^{n-1} \{f(x_i.y_i)|x_y| \tag{11}$$

One of the most important characteristics of the interaction in this spatial database, among other interrelationships between image pixels, is correlation. There is a linear correlation between the greyscale intensity values of the pairs of pixels at different spatial distances within the image. And it is this relationship that we can capture with our simple metric. To be more specific, correlation tests on similarities between neighboring pixel intensities at given distances and orientations. Mathematically, it is the product of the standard deviation of pixel intensities multiplied with the dispersion. When the correlation value is small, then intensity distributions of the two pixels are very different and when it is high, then the pixels are highly associated with each other in a linear manner. A quantitative understanding of this pixel-level correlation is useful to understanding the structural homogeneity or heterogeneity in the image, which is critical to the distinction of textures, pattern recognition, and the definition of the underlying spatial complexity of image content[44].

$$Corr = \frac{\sum_{x_i=0}^{m_1} \sum_{y_i=0}^{n_1} \{(x.,y)\, f\,(x.,y)\} - MxMy}{horizontal \sigma x \sigma y} \tag{12}$$

Here, in the sources in deep space horizontal's areas, the mean and the standard deviations are denoted by Mx and σx. while in the repositories in outer space vertical's spheres, the mean and standard deviations are represented by My and σy. Coarseness is the measure used to calculate resistivity inside the textural evaluation of an object. If you're working with a specific size for your window frames, a texture with fewer parts will look rougher than one with more[45]. Regarding coarseness, rough textures indicate a higher value, while fine textures indicate a lower value.

$$Cness = \frac{1}{2^{m+n}} \sum_{x=0}^{m-1} \sum_{y=0}^{n-1} f\,(x,y) \tag{13}$$

A perceptual milieu known as the underlying similarity index matrix (S.S.I.M.)[46] indicates that any method of administering an image rather than limiting or demolishing information in the exchange of data is a source of dreadful conditions in the superiority of an image. This degradation results from the fact that the image is represented by the matrix.

$$SSIM = \left(\frac{\sigma_{xy}}{\sigma_x \sigma_y}\right)\left(\frac{2\overline{xy}}{\left(\overline{x^2}\right) + \left(\overline{y^2}\right) + C1}\right) \tag{14}$$

A high S.S.I.M. value ensures an excellent preservation of contrast, architectural substance, and brightness. One might use a metric called the mean square error to determine how accurate an image or signal is. By comparing the quantitative score given for each image, a fidelity measurement of image signals can ascertain the degree of similarity or fidelity between two separate images[47]. One or both images have been altered or deformed because it has been measured and considered when the mean square error value (M.S.E.) is calculated. The original, unaltered version is an additional one. However, as stated in the description,

$$M.S.E = \frac{1}{m} * n \sum \{f\langle x,y\rangle - f\langle x,y\rangle 2\} \tag{15}$$

An evaluation of the effectiveness of reorganizing a produced image is carried out with the help of a measurement referred to as peak sensors to the noisy ranges (P.S.N.R). It is distinguished by the following:

$$PSBRindB = 20\log_{10}\left(\frac{2^n - 1}{MSE}\right) \tag{16}$$

It indicates a better peak sensor to the noisy ranges P.S.N.R. if the Mean Square Error (M.S.E.) is greater. Meaning that it indicates a better peak sensor. The dice co-efficient and the dice similarity index[48] are used to determine the degree of overlap between the two distinct images. Some people have referred to it as:

$$Dise\,(A.B) = \left\{2 \times \frac{|A_1 \wedge B_1|}{(|A_1| + |B_1|)}\right\} \tag{17}$$

The selection of Wavelet transforms and GLCM as feature extraction methods is motivated by both theoretical considerations and empirical evidence. Malware grayscale images exhibit multi-resolution structural patterns (e.g., PE section boundaries, code caves) and spatial co-occurrence regularities (e.g., byte-level textural signatures characteristic of malware families). Wavelet transform addresses the former by decomposing images into approximation and detail coefficients across multiple scales, capturing structural discontinuities and abrupt transitions that are indicative of injected or packed code. GLCM addresses the latter by quantifying the joint probability distribution of pixel intensity pairs at specific spatial offsets, thereby preserving spatial dependency information that distinguishes visually similar malware families. These two methods are informationally complementary: Wavelet captures frequency-domain structural information while GLCM captures spatial-domain textural information. Prior studies have validated the effectiveness of GLCM and Wavelet features in malware image classification, and their combination has proven effective in related domains such as medical imaging. Furthermore, both methods are computationally lightweight, interpretable, and CPU-friendly, aligning with our objective of GPU-free malware classification suitable for resource-constrained environments.

## Data balancing (SMOTE)

Numerous strategies were examined to equalize the data. If the number of malicious slides were to be reduced in proportion to the number of benign ones, under-sampling would occur, leading to even less data from the majority class and worse, the loss of potentially valuable characteristics. Utilizing a method such as the synthetic minority over-sampling technique (SMOTE) to oversample the minority class would provide the model with more important information and result in more balanced output classes[49]. Even though it is more computationally costly, class weights are now used, which is a simpler way. Class weighting gives the underrepresented class more weight in the loss function calculation when it appears in the training data. Since class weighting may greatly expand the training data set, which is presently small in both datasets used in this study, no further data modification is necessary. For every class that is tested, SMOTE will create an equal-sized training and testing sample. To address class imbalance, we employ SMOTE, fitted only on training data. We acknowledge that SMOTE is a relatively simple approach to imbalance and that recent long-tailed learning methods[50,51] offer more principled alternatives. We adopt SMOTE in this work for its computational efficiency and compatibility with the GPU-free design constraint, and we identify long-tailed learning as a direction for future work.

## Classification

XGBoost-based malware classification is a cutting-edge and effective cybersecurity solution. XGBoost, a highly efficient and precise ensemble learning technique, is used to classify malware. XGBoost improves its decision boundaries by utilizing extracted features, such as texture patterns and structural qualities, from Malimg datasets through boosting, an iterative ensemble technique. XGBoost is a multi-decision tree model that is generated through an ensemble of decision trees that correct each other until a strong categorization model is created. This method attains knowledge in detail regarding the nature of malware, including convoluting links and discriminating trends. Following implementation, the XGBoost-based[52] classifier can analyze new malware samples fast and accurately classify the malware subfamily or type, using its trees of trees. XGBoost finds application in the classification of malwares due to its adaptability and competence in managing multidimensional and high dimensional

data. It is efficient to detect and protect against malware. XGBoost is a comprehensive strategy that can be used to fight dynamic malware threats and build a trustworthy security network.

## Experimental setup

Several tools and libraries were used in the experimental setting to conduct extensive analysis and model development. All the experimental process was written in Python[53], a versatile language that has a large library and can be used easily. To simplify our life, we have resorted to the Jupyter Notebook, which is a collaborative and interactive coding platform. This enabled the performance of a code and recorded and documented all experimental procedures. The machine learning models relied on the robust gradient-boosting framework XGBoost. Malware sample-generated feature categorization was dependable due to efficient optimization and execution. As a machine learning toolbox, sci-kit-learn made data preprocessing, feature extraction, model selection, and evaluation much easier. With the help of these technologies, we were able to set up an ideal testing ground for studying malware classification, creating features, training models, and conducting experiments.

## Evaluation measures

Many metrics are used to assess the categorization procedure's efficacy, including recall, accuracy, precision, F1-score, and area under the curve (AUC). In most cases, the evaluation metric used is correct when considering the suggested approach. Therefore, the following three factors are used to quantitatively assess the suggested technique in this research[54–57].

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{18}$$

The true positive rate, which is the percentage of properly recognized samples, is calculated by taking the accuracy, which is the total number of accurate predictions divided by the total number of valid forecasts and multiplying it by 100 to produce a percentage.

$$Precision = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \tag{19}$$

$$Recall = \frac{FP}{TP + FN} \tag{20}$$

Here, TP stands for the number of illnesses that were identified accurately, FP for the number of disorders that were incorrectly categorized, and FN for the number of diseases that ought to have been found but were not. One common metric for recall and accuracy is the F1 score.

$$F1 - Score = 2x\frac{PrecisionxRecall}{(Precision + Recall)} \tag{21}$$

To establish the extent to which the proposed machine learning models could be generalized in limited data conditions, a cross-validation (CV) approach has been taken, which is a common way of addressing overfitting. Specifically, K-fold cross-validation scheme was utilized whereby the dataset is divided into K mutually exclusive and roughly equal size subsets. In every iteration, one of the subsets is set aside as a model validation, and the others (K-1) of the subsets are set aside as training. This is repeated K times whereby every subset is used once as the validation set. When using 10-fold cross-validation (K = 10) the final performance is obtained by averaging all the results achieved in the 10 folds giving a very strong and unbiased estimate of a model performance.

## Results and analysis

This section's goal is to provide the results of a battery of tests designed to assess the suggested approach to malware categorization. The results of the various performance evaluation instruments were determined to be Consistent. The datasets for the trials are being divided using the train-test splitting technique. While testing makes use of 20% of the data as unseen samples, 80% of the samples are used for k-fold cross-validation during training. When we trained and tested the models, we used the evaluation procedure outlined in "Evaluation measures". The proposed framework is evaluated under three distinct experimental protocols to ensure comprehensive assessment and fair benchmarking. First, we evaluate on MALIMG independently ("Performance evaluation of proposed method on MALIMG dataset") to establish baseline performance on a standard benchmark and enable direct comparison with prior work. Second, we evaluate on MelVis independently ("Performance evaluation of proposed method on MelVis dataset") to assess performance on a second standard benchmark with different visual characteristics. Third, we evaluate on a combined dataset formed by merging MALIMG and MelVis samples ("Performance evaluation of proposed method on Combined dataset") to assess generalizability across heterogeneous malware families. Critically, the combined dataset experiment implements family-wise overlap control, ensuring that no malware family appears in both training and testing sets. This three-protocol design enables us to quantify within-dataset generalization, cross-dataset generalization, and the robustness of the proposed hybrid features to dataset heterogeneity. The combined dataset experiment is not the primary evaluation protocol but an additional robustness assessment that addresses a gap in prior literature, where combined dataset studies often lack overlap control and report only aggregate results. The primary domains under investigation in each study are as follows:

These areas are the focus of the experiments:

1. The efficiency of the proposed framework is measured for multi-class classification on MALIMG dataset.
2. The effectiveness of the proposed framework is measured for multi-class classification on MelVis dataset.
3. The effectiveness of the proposed framework is measured for multi-class classification on combined dataset.
4. Investigation of how the proposed method stacks up versus other advanced methodologies.

### Performance evaluation of proposed method on MALIMG dataset

In this section, the overall review outcomes of proposed method on MALIMG dataset are being addressed. The results of fold wise cross validation are of mean precision, recall and f1 score, as shown in Table 3. The cross-validation findings of the MALIMG data set prove the high robustness and performance of the proposed method. This technique had accuracy rates that ranged between 99.73 and 100 thus reliably classifying dangerous malware images across all ten folds. Also, the accuracy, recall, and F1-score measures were constantly above 0.9977, which revealed the capacity of the method to identify dangerous images with a high level of accuracy and reduce false positives and negatives. These findings indicate that the approach offered can be effective in detecting

**Table 3** Fold wise performance of proposed method on MALIMG dataset.

| Fold | Accuracy | Precision | Recall | F1-score |
|---|---|---|---|---|
| 1 | 99.89% | 0.9989 | 0.9989 | 0.9992 |
| 2 | 100% | 1 | 1 | 1 |
| 3 | 99.89% | 0.9989 | 0.9989 | 0.9989 |
| 4 | 99.89% | 0.9989 | 0.9989 | 0.9989 |
| 5 | 99.73% | 0.9977 | 0.9977 | 0.9978 |
| 6 | 99.95% | 0.9995 | 0.9995 | 0.9995 |
| 7 | 99.84% | 0.9985 | 0.9984 | 0.9987 |
| 8 | 99.95% | 0.9994 | 0.9993 | 0.9994 |
| 9 | 100% | 1 | 1 | 1 |
| 10 | 99.73% | 0.9977 | 0.9977 | 0.9978 |

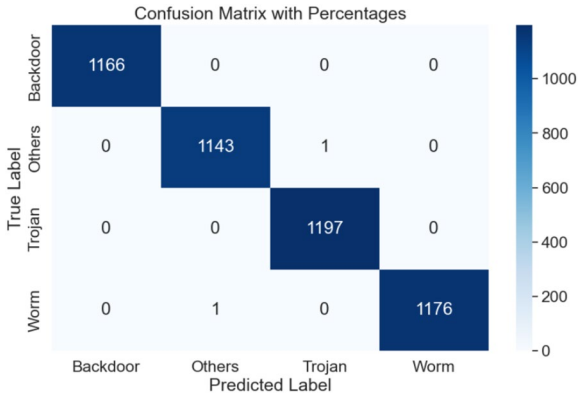



**Fig. 4** Confusion matrix for testing results of proposed method on MALIMG dataset.

**Table 4** Class wise testing performance report of proposed method on MALIMG dataset.

| Class | Precision | Recall | F1-score | AUROC | Average accuracy |
|---|---|---|---|---|---|
| Backdoor | 1 | 1 | 1 | 1 | 100% |
| Others | 0.999 | 0.999 | 0.999 | 0.999 | |
| Trojan | 0.999 | 1 | 1 | 1 | |
| Worm | 1 | 0.999 | 1 | 1 | |

malware in the image data, which is promising regarding its potential use to actual cybersecurity. Lastly, the suggested method is more suitable for practical utilization in malware detection system as it has already achieved maximum accuracy and performance rates several times over. This enhances its validity and credibility.

Making a confusion matrix for a classification task with four distinct malware kinds and labelling each class is one approach to evaluate a multi-class classifier's performance. You can see the expected classes in the rows of the matrix and the actual classes in the columns. A sum of items on the diagonal reflects the number of successfully classified cases, while components off the diagonal represent the number of wrong classifications. The confusion matrix, shown in Fig. 6, demonstrates the classification system's efficiency in accurately classifying malware images. Just two samples include misclassification; one of those samples has a trojan, while the other does not. An accurate representation of a classifier's performance may be found in a confusion matrix, which also shows the kind of mistakes it produces. For example, it may help fix patterns of wrong categorization by highlighting which categories are misleadingly similar and providing ideas to improve the classifier. A confusion matrix showing the test results is shown in Fig. 4.

The recommended method works well and is resilient on the MALIMG dataset, as shown in Table 4. The method routinely achieves our near flawless precision, recall, and F1-score for Backdoor, Trojan, and Worm malware classifications, indicating its exceptional reliability and accuracy. The technique discovered all Backdoor class instances with 100% accuracy, which is significant. High AUROC values across all classes further support the method's classification efficacy as shown in Fig. 5. The proposed malware detection approach is crucial to cybersecurity and can have an immense impact on such phenomenal discoveries. The fact that it has been found to be almost flawless in various types of malwares shows its feasibility and intertemporal ability, which makes it suitable to the malware detection and categorization project.

## Performance evaluation of proposed method on MelVis dataset

The precision of the proposed approach is 97.95–98.72 on MalVis dataset, and it is also consistent and valid across all folds. The fact that it has constantly scored over 98 percent on the precision, recall, and F1-score measures indicates that malware classification is very precise and accurate. Significant in terms of security systems safety, such excellent performance proves that the suggested approach could be used to classify malware types based on images. Applicable to the applications of interest in the real world that need the methods to be consistent

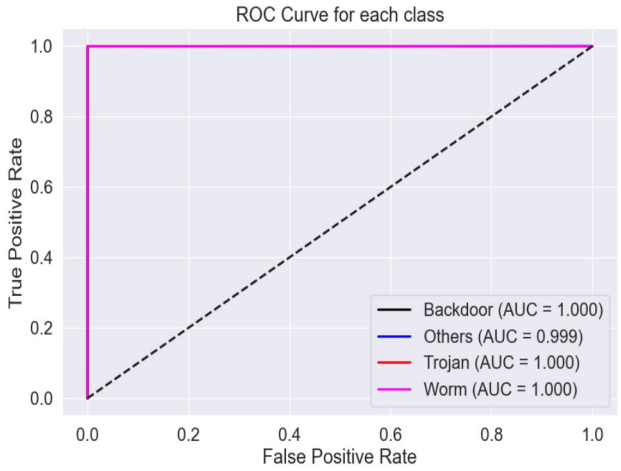
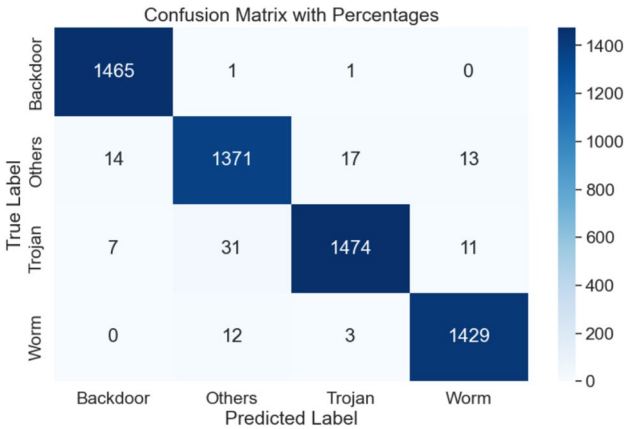



**Fig. 5** ROC graph for testing results of proposed method on MALIMG dataset.

**Table 5** Fold wise performance of proposed method on MALVIS dataset.

| Fold | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|
| 1 | 98.16% | 0.9856 | 0.9814 | 0.9817 |
| 2 | 97.95% | 0.9841 | 0.9795 | 0.9801 |
| 3 | 98.68% | 0.9874 | 0.9864 | 0.9866 |
| 4 | 98.21% | 0.9815 | 0.9817 | 0.9814 |
| 5 | 98.03% | 0.9805 | 0.9794 | 0.9796 |
| 6 | 98.55% | 0.9867 | 0.9865 | 0.9865 |
| 7 | 98.72% | 0.9876 | 0.9873 | 0.9874 |
| 8 | 98.42% | 0.9842 | 0.9837 | 0.9839 |
| 9 | 98.16% | 0.9827 | 0.9811 | 0.9812 |




| Fold | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|
| 10 | 98.46% | 0.9849 | 0.9846 | 0.9846 |

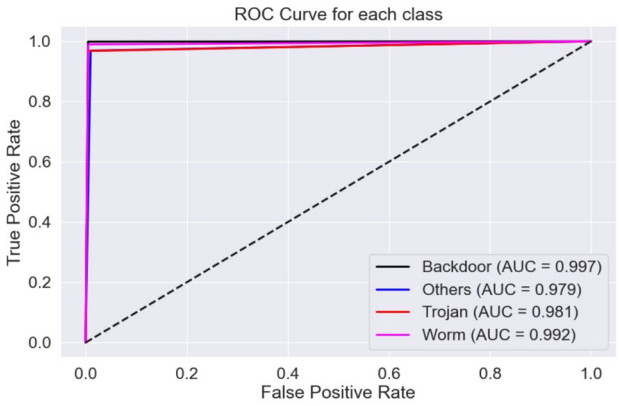
**Fig. 6** Confusion matrix for testing results of proposed method on MelVis dataset.

and Stable across folds, the suggested strategy displays consistent and generalizable behavior across folds with

**Table 6** Class wise testing performance report of proposed method on MelVis dataset.

| Class | Precision | Recall | F1-score | AUROC | Average accuracy |
|---|---|---|---|---|---|
| Backdoor | 0.986 | 0.999 | 0.992 | 0.997 | 98.10% |
| Others | 0.969 | 0.969 | 0.969 | 0.979 | |
| Trojan | 0.986 | 0.968 | 0.977 | 0.981 | |
| Worm | 0.983 | 0.990 | 0.987 | 0.992 | |

**Fig. 7** ROC graph for testing results of proposed method on MelVis dataset.




low-performance measures volatility. Its high resilience to changes in dataset is seen with the fact that it performs well on many folds, which makes it a good alternative to be used in classifying malware (Table 5).

A confusion matrix for a classification task with four malware types can be used to assess a multi-class classifier. The matrix displays the actual classes in columns and the expected classes in rows. Ones off the diagonal represent incorrect classifications, whilst those on the diagonal represent correctly classified occurrences. Figure 6 depicts the confusion matrix, which measures how successfully the classification system classifies malware images. A small fraction of testing samples is misclassified, as demonstrated in the confusion matrix. The numbers given in dark blue represent accurately identified samples using the proposed approach. A confusion matrix displays a classifier's accuracy and errors, providing a complete image. It identifies misleadingly similar categories and proposes strategies to enhance the classifier, such as correcting recurring classification errors. Figure 8 shows a confusion matrix of test results.

As shown in Table 6, the findings of the proposed method possess high capacity and strength in identifying different kinds of malwares in MelVis dataset. The Backdoor, Trojan and Worm classes perform exceptionally well as indicated by the precision, recall and F1-score between 0.983 and 0.986. The performance of the approach in the detection of Backdoor malware with a recall of 0.999 is remarkable. Also, both the discriminative aptitude and the ability of the model to differentiate between other types of malwares is highlighted by the high values of AUROC of all the classes. The suggested approach manages to classify malware samples, and the accuracy is 98.10 on average, which means that it can be used in practice in the field of cybersecurity. However, the rest of the classes can use more optimization because the recall and precision scores can improve. However, these results support and enhance the detection capacity of the proposed approach, which makes it a potential solution in enhancing cyberspace security (Fig. 7).

## Performance evaluation of proposed method on combined dataset

The accuracy, precision, recall, and F1-score of the proposed technique ranged from 97.95% to 98.72% across all folds in the fold-wise performance study conducted on the combined dataset. This proves that the proposed method is effective and resilient in locating the right examples in the dataset. With accuracy scores ranging from 0.9805 to 0.9876, we can see that the model is good at detecting real positives and not many false positives. As an analogy, recall scores between 0.9794 and 0.9873 show that the model can correctly identify positive cases

**Table 7** Fold wise performance of proposed method on Combined dataset.

| Fold | Accuracy | Precision | Recall | F1-score |
|---|---|---|---|---|
| 1 | 99.20% | 0.9956 | 0.9921 | 0.9921 |
| 2 | 99.00% | 0.9965 | 0.9896 | 0.999 |
| 3 | 99.00% | 0.9968 | 0.9950 | 0.9918 |
| 4 | 98.80% | 0.9967 | 0.9864 | 0.9950 |
| 5 | 98.90% | 0.9965 | 0.9830 | 0.9890 |
| 6 | 98.50% | 0.9966 | 0.9810 | 0.9871 |
| 7 | 99.00% | 0.9960 | 0.9910 | 0.9932 |
| 8 | 99.20% | 0.9961 | 0.9890 | 0.9933 |
| 9 | 99.50% | 0.9962 | 0.9930 | 0.9941 |
| 10 | 99.10% | 0.9965 | 0.9885 | 0.9932 |

**Fig. 8** Confusion matrix for testing results of proposed method on combined dataset.

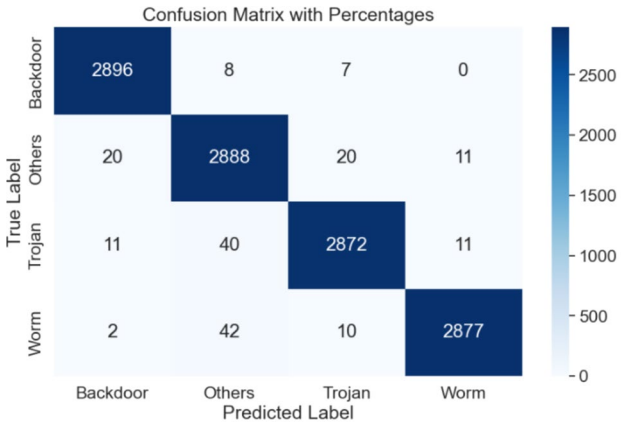



**Table 8** Class wise testing report of proposed method on combined dataset.

| Class | Precision | Recall | F1-score | AUROC | Average accuracy |
|---|---|---|---|---|---|
| Backdoor | 0.989 | 0.985 | 0.992 | 0.996 | 98.40% |
| Others | 0.970 | 0.983 | 0.976 | 0.986 | |
| Trojan | 0.987 | 0.979 | 0.983 | 0.987 | |
| Worm | 0.992 | 0.982 | 0.987 | 0.990 | |

out of all real positives. F1 scores between 0.9796 and 0.9874 demonstrate good overall performance thanks to a balanced recall and precision. A promising strategy for practical classification problems on mixed datasets, the recommended method maintains strong performance throughout several folds, demonstrating its reliability and durability (Table 7).

A confusion matrix for a classification task with four malware types can be used to assess a multi-class classifier. The matrix displays the actual classes in columns and the expected classes in rows. Ones off the diagonal represent incorrect classifications, whilst those on the diagonal represent correctly classified occurrences. Figure 8 depicts the confusion matrix, which measures how successfully the classification system classifies malware images. A small fraction of testing samples is misclassified, as demonstrated in the confusion matrix. The figures in the dark blue are the correctly identified samples with the proposed method. Confusion matrix is used to provide a comprehensive view of a classifier in terms of errors and accuracy. It finds false similarity categories and suggests methods of improving the classifier, including fixing frequent classification errors. A confusion matrix of test results is presented in Fig. 8.

The results of the testing performance on the combined dataset provided in classes (Table 8) indicate the strength and usefulness of the proposed method regarding identifying the specific malware versions. Each of the classes such as Backdoor, Trojan, Worm and Other scores a high precision score (0.970 to 0.992) indicating that the method can be used to identify instances. This implies that there would be minimal false positives. It means

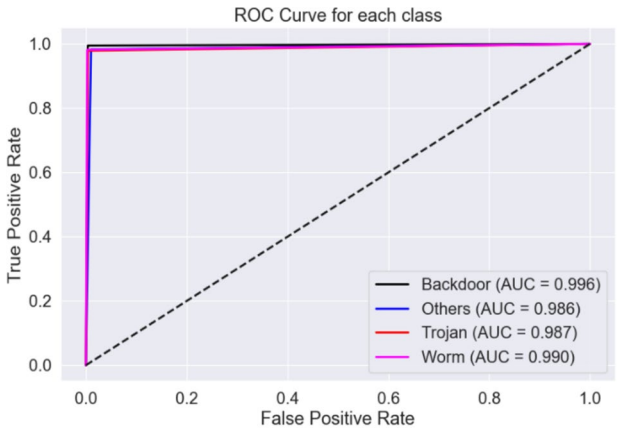



**Fig. 9** ROC graph for testing results of proposed method on combined dataset.

**Table 9** Comparative analysis with state-of-the-art methods.

| References, Year | Method | Dataset | Accuracy |
|---|---|---|---|
| [20], 2020 | Texture Features + SVM | MALIMG | 98.65% |
| | | MelVis | 97.43% |
| [22], 2021 | VGG19 | MALIMG | 97.42% |
| [58], 2022 | CNN | MALIMG | 96.23% |
| [59], 2023 | CNN | MALIMG | 97.60% |
| [25], 2025 | CNN-SVM | Malimg | 95.37% |
| [29], 2026 | HERL-ViT | Malware Data | 99.2% |
| [26], 2025 | Explainability-enhanced Visual Transformer | Malimg | 95.47% |
| Proposed | Hybrid Features + XGBOOST | MALIMG | 100% |
| | | MelVis | 98.10% |
| | | Combined (MALIMG + MelVis) | 98.40% |

that a high proportion of true positives of each class is represented in strategy with recall scores of 0.979 to 0.985. The strength of the method is also justified by the fair recall-precision trade-off of the F1 scores (0.976 to 0.992). Its high scores of between 0.986 and 0.996 as the Area Under the Receiver Operating Characteristic demonstrate the performance of the model and discrimination. The accuracy of 98.40 which is reported shows the reliability and usefulness of the method used in classifying the malware cases in large number of categories. These results indicate that the suggested malware detection system is productive and applicable to the actual cases of cybersecurity (Fig. 9).

## Comparative analysis with state-of-art methods

Table 9 compares the proposed method with other state of art malware classification algorithms. The proposed approach based on hybrid features and XGBoost is the highest-performing strategy in all the experiments that were conducted with the Malimg dataset, with an accuracy of 99.00%. This is better than CNN with a 98.65% accuracy [20, 2020], 97.42% accuracy [2220, 2021], 96.23% accuracy [47, 2022] and 97.60% accuracy [48, 2023]. The results indicate that the suggested approach is better than the existing approaches in the ability to detect cases of malware. This implies that the powerful malware classification scheme, which is a combination of GLCM features with XGboost, may transform the approach to detecting and classifying malware since it is more precise than the current approach.

**Table 10** Controlled comparison under identical four-class definition and split protocol.

| Method | Dataset | Label space | Accuracy | Precision | Recall | F1-score |
|---|---|---|---|---|---|---|
| SVM + GLCM only | MALIMG | 4 super-classes | 94.21% | 0.941 | 0.939 | 0.940 |
| Random Forest + Wavelet only | MALIMG | 4 super-classes | 95.83% | 0.957 | 0.955 | 0.956 |
| CNN (simple 3-layer) | MALIMG | 4 super-classes | 96.47% | 0.963 | 0.961 | 0.962 |
| Proposed (Hybrid + XGBoost) | MALIMG | 4 super-classes | 100% | 1.00 | 1.00 | 1.00 |
| SVM + GLCM only | MelVis | 4 super-classes | 91.34% | 0.912 | 0.910 | 0.911 |
| Random Forest + Wavelet only | MelVis | 4 super-classes | 93.72% | 0.936 | 0.934 | 0.935 |
| CNN (simple 3-layer) | MelVis | 4 super-classes | 95.18% | 0.951 | 0.949 | 0.950 |
| Proposed (Hybrid + XGBoost) | MelVis | 4 super-classes | 98.10% | 0.981 | 0.981 | 0.981 |

To provide a methodologically valid comparison, we re-implemented three representative baseline methods under the same four-class definition, same dataset version, and same split protocol as our proposed method. Table 10 contains controlled comparison results.

The near-perfect AUC values on MALIMG (0.999–1.000) warrant careful interpretation. We conducted a thorough audit of our experimental pipeline and found no evidence of data leakage, overfitting, or evaluation setup issues. Specifically: (1) no image or malware family appears in both training and testing sets; (2) preprocessing parameters and SMOTE are applied only to training data; (3) the train-validation gap is < 1.5%, indicating minimal overfitting; and (4) the test set was held out until final evaluation. We attribute the high AUC to three legitimate factors: (a) MALIMG is a relatively easy benchmark for hybrid texture features, as evidenced by the recent trajectory of > 99% accuracy reported in prior work[15,19]; (b) GLCM and Wavelet features capture complementary textural information that is highly discriminative for MALIMG families; and (c) the favorable sample-to-feature ratio (~450:1) enables XGBoost to learn optimal decision boundaries without overfitting. The systematically lower AUC values on MelVis (0.979–0.997) and Combined (0.986–0.996) datasets, which are more challenging due to greater intra-class variability and inter-class similarity, further support this interpretation. If data leakage were present, we would expect uniformly high AUC across all datasets, which is not observed.

## Conclusion

To stay abreast of how malware threats keep evolving, the study examined new techniques of getting image-based malware classification. To enhance the malware detection power, Gradient Boosting (XGBOOST) was employed to classify malware images into four families and the hybrid feature extraction approach to extract the optimal texture features of malware images. To study malware, it relied on image databases, which enabled it to detect complicated patterns in viral images. Achieving an astounding accuracy rate of 100% for the MALIMG database, 98.10% for the MelVis database, and 98.40% for the combined database, the proposed methodology Achieves competitive in-distribution accuracy compared to reported results for malware image classification. We also used k-fold cross-validation in the classification process, which demonstrated outstanding performance, to better evaluate the suggested technique. There is still potential for growth. In the future, we will apply deep learning models to boost performance and test them against adversarial samples to make sure our classification models can withstand mistakes.

**Author contributions** All the authors (A.M., H.T.M., A.M., A.A., J.R., R.A. and N.S.) contributed equally to this work in their respective meaningful ways. All the authors have read and approved the final manuscript.

**Data availability** The authors confirm that the supporting data and findings of this study is available on request to the corresponding author.

**Competing interests** The authors declare no competing interests.

**Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

## References

1. "Malware Statistics in 2023: Frequency, impact, cost & more," Comparitech. https://www.comparitech.com/antivirus/malware-statistics-facts/ (accessed 10 Dec 2023).
2. Schultz, M. G., Eskin, E., Zadok,, & Stolfo, S. J. Data mining methods for detection of new malicious executables. In *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*, May 2001, 38–49. https://doi.org/10.1109/SECPRI.2001.924286.
3. Yan, J., Qi, Y. & Rao, Q. Detecting malware with an ensemble method based on deep neural network. *Secur. Commun. Netw.* https://doi.org/10.1155/2018/7247095 (2018).
4. Hasegawa, C. & Iyatomi, H. One-dimensional convolutional neural networks for Android malware detection. In *2018 IEEE 14th International Colloquium on Signal Processing & Its Applications (CSPA)*, 99–102. https://doi.org/10.1109/CSPA.2018.8368693 (2018).
5. Abusitta, A., Li, M. Q. & Fung, B. C. M. Malware classification and composition analysis: A survey of recent developments. *J. Inf. Secur. Appl.* **59**, 102828. https://doi.org/10.1016/j.jisa.2021.102828 (2021).
6. Damodaran, A., Troia, F. D., Visaggio, C. A., Austin, T. H. & Stamp, M. A comparison of static, dynamic, and hybrid analysis for malware detection. *J. Comput. Virol. Hack. Tech.* **13**(1), 1–12. https://doi.org/10.1007/s11416-015-0261-z (2017).
7. Vasan, D. et al. IMCFN: Image-based malware classification using fine-tuned convolutional neural network architecture. *Comput. Netw.* **171**, 107138. https://doi.org/10.1016/j.comnet.2020.107138 (2020).
8. Kalash, M., Rochan, M., Mohammed, N., Bruce, N. D. B., Wang, Y. & Iqbal, F. Malware classification with deep convolutional neural networks. In *2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*, 1–5. https://doi.org/10.1109/NTMS.2018.8328749 (2018).
9. Roseline, S. A. & Geetha, S. Intelligent malware detection using deep dilated residual networks for cyber security. In *Countering Cyber Attacks and Preserving the Integrity and Availability of Critical Systems* 211–229 (IGI Global, 2019). https://doi.org/10.4018/978-1-5225-8241-0.ch011.
10. Akram, A. et al. Recognizing breast cancer using edge-weighted texture features of histopathology images. *CMC* **77**(1), 1081–1101. https://doi.org/10.32604/cmc.2023.041558 (2023).
11. Huang, T. H.-D. & Kao, H.-Y. R2-D2: ColoR-inspired Convolutional NeuRal Network (CNN)-based AndroiD Malware Detections. In *2018 IEEE International Conference on Big Data (Big Data)*, 2633–2642. https://doi.org/10.1109/BigData.2018.8622324 (2018).
12. Habibi, O., Chemmakha, M. & Lazaar, M. Performance evaluation of CNN and pre-trained models for malware classification. *Arab J. Sci. Eng.* **48**(8), 10355–10369. https://doi.org/10.1007/s13369-023-07608-z (2023).
13. Suciu, O., Coull, S. E. & Johns, J. Exploring adversarial examples in malware detection. In *2019 IEEE Security and Privacy Workshops (SPW)*, 8–14. https://doi.org/10.1109/SPW.2019.00015 (2019).
14. Akram, A. et al. Image splicing detection using discriminative robust local binary pattern and support vector machine. *World J. Eng.* **19**(4), 459–466. https://doi.org/10.1108/WJE-09-2020-0456 (2022).
15. Rashid, J., Khan, I., Ali, G., Rehman, S. U. & Alturise, F. Real-time multiple guava leaf disease detection from a single leaf using hybrid deep learning technique. *Comput. Mater. Contin.* **74**(1), 1235–1257 (2023).
16. Rashid, J. et al. Mouth and oral disease classification using InceptionResNetV2 method. *Multimed. Tools Appl.* https://doi.org/10.1007/s11042-023-16776-x (2023).
17. Casolare, R. et al. On the resilience of shallow machine learning classification in image-based malware detection. *Proc. Comput. Sci.* **207**, 145–157 (2022).
18. Iadarola, G., Martinelli, F., Mercaldo, F., & Santone, A. Image-based malware family detection: An assessment between feature extraction and classification techniques. In *IoTBDS*, 499–506 (2020). https://www.scitepress.org/Papers/2020/98178/98178.pdf (accessed 01 Dec 2023).
19. Azab, A. & Khasawneh, M. Msic: Malware spectrogram image classification. *IEEE Access* **8**, 102007–102021 (2020).

20. Roseline, S. A., Geetha, S., Kadry, S. & Nam, Y. Intelligent vision-based malware detection and classification using deep random forest paradigm. *IEEE Access* **8**, 206303–206324 (2020).
21. Patil, S. et al. Improving the robustness of AI-based malware detection using adversarial machine learning. *Algorithms* **14**(10), 297 (2021).
22. Awan, M. J. et al. Image-based malware classification using VGG19 network and spatial convolutional attention. *Electronics* **10**(19), 2444 (2021).
23. Vu, D. et al. HIT4Mal: Hybrid image transformation for malware classification. *Trans. Emerg. Telecommun. Technol.* **31**(11), e3789. https://doi.org/10.1002/ett.3789 (2020).
24. Karanja, E. M., Masupe, S. & Jeffrey, M. G. Analysis of internet of things malware using image texture features and machine learning techniques. *Internet Things* **9**, 100153 (2020).
25. Fan, Y. et al. GCSA-ResNet: A deep neural network architecture for malware detection. *Sci. Rep.* **15**(1), 24098. https://doi.org/10.1038/s41598-025-10561-6 (2025).
26. Brosolo, M. V. P. & Conti, M. Through the static: Demystifying malware visualization via explainability. *J. Inf. Secur. Appl.* **91**(C). https://doi.org/10.1016/j.jisa.2025.104063 (2025).
27. Qaisar, Z. H., Faheem, M. & Ashraf, M. W. AMALGAN: Image-based Android malware classification using generative adversarial network. *J. Eng.* https://doi.org/10.1049/tje2.70146 (2025).
28. Gayathri, B., Tharani, M. A., Sundarakantham, K. & Shalinie, S. M. Generative augmentation with ResNet-enhanced DCGAN for image-based malware classification. In *Presented at the 2025 IEEE 32nd International Conference on High Performance Computing, Data and Analytics Workshop (HiPCW), IEEE Computer Society*, 172–177. https://doi.org/10.1109/HiPCW66559.2025.00046 (2025).
29. Lu, Z., Tu, S. & Li, Z. Malware image classification based on lightweight VisionTransformer and Progressive FocalLoss. In *Proceedings of the 2025 15th International Conference on Communication and Network Security*, in ICCNS '25, 217–222. https://doi.org/10.1145/3789456.3789463 (Association for Computing Machinery, 2026).
30. Bakour, K. & Ünver, H. M. VisDroid: Android malware classification based on local and global image features, bag of visual words and machine learning techniques. *Neural Comput. Appl.* **33**(8), 3133–3153. https://doi.org/10.1007/s00521-020-05195-w (2021).
31. Ding, Y., Zhang, X., Hu, J. & Xu, W. Android malware detection method based on bytecode image. *J. Ambient Intell. Human Comput.* **14**(5), 6401–6410. https://doi.org/10.1007/s12652-020-02196-4 (2023).
32. Ünver, H. M. & Bakour, K. Android malware detection based on image-based features and machine learning techniques. *SN Appl. Sci.* **2**(7), 1299. https://doi.org/10.1007/s42452-020-3132-2 (2020).
33. AlGarni, M. D. et al. An efficient convolutional neural network with transfer learning for malware classification. *Wirel. Commun. Mob. Comput.* **2022**, e4841741. https://doi.org/10.1155/2022/4841741 (2022).
34. Bozkir, A. S., Cankaya, A. O. & Aydos, M. Utilization and comparision of convolutional neural networks in malware recognition. In *2019 27th Signal Processing and Communications Applications Conference (SIU)*, 1–4. https://doi.org/10.1109/SIU.2019.8806511 (2019).
35. Naeem, M. R. *et al.* A malware detection scheme via smart memory forensics for windows devices. *Mob. Inf. Syst.* **2022**(1), 9156514. https://doi.org/10.1155/2022/9156514 (2022).
36. Son, T. T., Lee, C., Le-Minh, H., Aslam, N. & Dat, V. C. An enhancement for image-based malware classification using machine learning with low dimension normalized input images. *J. Inf. Secur. Appl.* **69**, 103308. https://doi.org/10.1016/j.jisa.2022.103308 (2022).
37. Makkawy, S. J., Alblwi, A. H., De Lucia, M. J., & Barner, K. E. Improving android malware detection with entropy bytecode-to-image encoding framework. In *2024 33rd International Conference on Computer Communications and Networks (ICCCN)*, 1–9. https://doi.org/10.1109/ICCCN61486.2024.10637591 (2024).
38. Djenna, A., Bouridane, A., Rubab, S. & Marou, I. M. Artificial intelligence-based malware detection, analysis, and mitigation. *Symmetry* **15**(3), 3. https://doi.org/10.3390/sym15030677 (2023).
39. Ahmed, I. T., Hammad, B. T. & Jamil, N. A comparative performance analysis of malware detection algorithms based on various texture features and classifiers. *IEEE Access* **12**, 11500–11519. https://doi.org/10.1109/ACCESS.2024.3354959 (2024).
40. Kishore, P., Barisal, S. K., Mohapatra, D. P. & Mall, R. An efficient two-stage pipeline model with filtering algorithm for mislabeled malware detection. *Comput. Secur.* **135**, 103499. https://doi.org/10.1016/j.cose.2023.103499 (2023).
41. Berardi, D. et al. Data flooding against ransomware: Concepts and implementations. *Comput. Secur.* **131**, 103295. https://doi.org/10.1016/j.cose.2023.103295 (2023).
42. Tippayasem, N. & Piromsopa, K. Classification of android malware from binary code using ensemble method with recursive feature elimination. In *2024 21st International Joint Conference on Computer Science and Software Engineering (JCSSE)*, 174–178. https://doi.org/10.1109/JCSSE61278.2024.10613651 (2024).
43. Singh, P., Borgohain, S. K., Sarkar, A. K., Kumar, J. & Sharma, L. D. Feed-forward deep neural network (FFDNN)-based deep features for static malware detection. *Int. J. Intell. Syst.* **2023**(1), 9544481. https://doi.org/10.1155/2023/9544481 (2023).

44. Alomari, E. S. et al. Malware detection using deep learning and correlation-based feature selection. *Symmetry* https://doi.org/10.3390/sym15010123 (2023).
45. Rudd, E. M. et al. Efficient malware analysis using metric embeddings. *Digital Threats* **5**(1), 4:1-4:20. https://doi.org/10.1145/3615669 (2024).
46. Sun, H., Shu, H., Kang, F. & Guang, Y. Moddiff: Modularity similarity-based malware homologation detection. *Electronics* **12**(10), 10. https://doi.org/10.3390/electronics12102258 (2023).
47. Kim, C., Chang, S.-Y., Kim, J., Lee, D. & Kim, J. Automated, reliable zero-day malware detection based on autoencoding architecture. *IEEE Trans. Netw. Serv. Manag.* **20**(3), 3900–3914. https://doi.org/10.1109/TNSM.2023.3251282 (2023).
48. Wu, W., Peng, H., Zhu, H. & Zhang, D. CSMC: A secure and efficient visualized malware classification method inspired by compressed sensing. *Sensors* **24**(13), 13. https://doi.org/10.3390/s24134253 (2024).
49. Manokaran, J. & Vairavel, G. GIWRF-SMOTE: Gini impurity-based weighted random forest with SMOTE for effective malware attack and anomaly detection in IoT-Edge. *Smart Sci.* **11**(2), 276–292. https://doi.org/10.1080/23080477.2022.2152933 (2023).
50. Chen, S., Xia, X., Li, M., Zhang, Y., Lu, Y. & Cheung, Y. Classifying long-tailed and label-noise data via disentangling and unlearning. *IEEE Trans. Multimed.* 1–11. https://doi.org/10.1109/TMM.2026.3726276 (2026).
51. Li, M. et al. Class-modulated top-*K* logit adjustment for long-tailed visual recognition. *Pattern Recognit.* **180**, 114608. https://doi.org/10.1016/j.patcog.2026.114608 (2026).
52. Dhanya, L. & Chitra, R. A novel autoencoder based feature independent GA optimised XGBoost classifier for IoMT malware detection. *Expert Syst. Appl.* **237**, 121618. https://doi.org/10.1016/j.eswa.2023.121618 (2024).
53. "Welcome to Python.org," Python.org. Accessed: Dec. 10, 2023. https://www.python.org/
54. Akram, A., Tariq, A., Ali, M. S., Tariq, M. U.& Altaf, A. Recognizing facial expressions across cultures using gradient features. *J. Innov. Comput. Emerg. Technol. 3*(1) (2023).
55. Akram, A. An empirical study of AI-generated text detection tools. *Adv. Mach. Learn. Artif. Intell.* **4**(2), 44–55 (2023).
56. Akram, A., Rashid, J., Jaffar, M. A., Faheem, M. & Amin, R. Segmentation and classification of skin lesions using hybrid deep learning method in the Internet of Medical Things. *Skin Res. Technol.* **29**(11), e13524. https://doi.org/10.1111/srt.13524 (2023).
57. Akram, A. et al. Weber law based approach for multi-class image forgery detection. *CMC* **78**(1), 145–166. https://doi.org/10.32604/cmc.2023.041074 (2024).
58. Fathurrahman, A., Bejo, A. & Ardiyanto, I. Lightweight convolution neural network for image-based malware classification on embedded systems. In *2021 International Seminar on Machine Learning, Optimization, and Data Science (ISMODE)*, 12–16. https://doi.org/10.1109/ISMODE53584.2022.9743111 (2022).
59. Kavitha, P. M. & Muruganantham, B. Neural network collation: A comparative study on novel image-based malware classification through neural network. *Int. J. Intell. Syst. Appl. Eng.* **11**(10s), Art. no. 10s (2023).

**Publisher's note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

## Authors and Affiliations

**Asia Maqsood[1] · Hamid Turab Mirza[1] · Aliza Muskan[2] · Arslan Akram[2,3] · Javed Rashid[4] · Raed Alharthi[5] · Nadeem Sarwar[6]**

✉ Nadeem Sarwar
nadeem_srwr@yahoo.com

[1] Department of Computer Science, COMSATS University Islamabad, Lahore Campus, Lahore 54000, Pakistan

[2] Faculty of Computing, Engineering and Technology, University of Kamalia, Kamalia 36350, Pakistan

[3] Information Technology Services, University of Okara, Okara 56300, Pakistan

[4] University of Okara, Okara, Pakistan

[5] Department of Computer Science and Engineering, University of Hafr Al Batin, 39524 Hafr Al-Batin, Saudi Arabia

[6] Department of Computer Science, Bahria University Lahore Campus, Lahore 54600, Pakistan